

Unit 03: Functions and Compile Time Polymorphism

CONTENTS

Objectives

Introduction

3.1 Why We Need Functions in C++

3.2 The Main Function

3.3 Function Types

3.4 Function Activities

3.5 Inline function

3.6 C++ Friend Functions

3.7 Characteristics of friend function

3.8 C++ Static Data Members & Functions

3.9 Polymorphism in C++

3.10 Types of C++ Polymorphism

3.11 Function Overloading

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Understand the functions
- Describe the function overloading
- Explain the inline functions
- Understand friend function
- Analyze C++ Static Data Members & Functions
- Define polymorphism in C++

Introduction

A function is a code module that only does one thing. Sorting, searching for a specific item, and inverting a square matrix are some instances. After a function is built, it is thoroughly tested. Following that, it is added to the library of functions. A user can use a library function as often as they like. This concept enhances software robustness while simultaneously shortening the time it takes to develop code. System-defined and user-defined functions are the two types of functions.

A function is a group of statements that together perform a task. Every C++ program has at least one function, which is `main()`, and all the most trivial programs can define additional functions.

A function declaration tells the compiler about a function's name, return type, and parameters. A function definition provides the actual body of the function.

3.1 Why We Need Functions in C++

- Debugging of the code would be easier if you use functions, as errors are easy to be traced.
- Use of functions reduce size of code.
- Duplicate set of statements are replaced by function calls.
- Improve reusability of code, same function can be used in any program.
- Use of function improves readability of code.

3.2 The Main Function

An application written in C++ may have a number of classes. One of these classes must contain one (and only one) method called main method. Although a private main method is permissible in C++ it is seldom used. For all practical purposes the main method should be declared as public method.

The main method can take various forms as listed below: Notes

main()

main(void)

void main()

void main(void)

int main()

int main(void)

int main(int argc, char *argv[])

main(int argc, char *argv[])

void main(int argc, char *argv[])

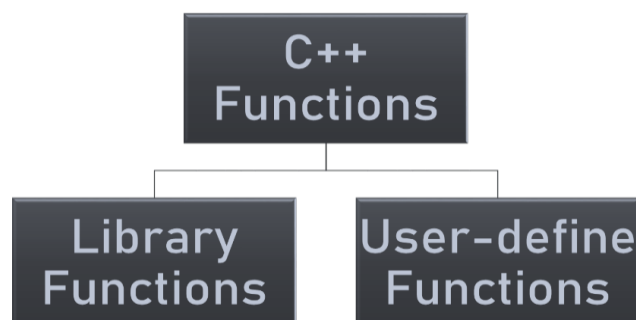
As is evident from the above forms, return type of the main method specifies the value returned to the operating system once the program finishes its execution. If your main method does not return any value to the operating system (caller of this method), then return type should be specified as void.



Caution: In case you design your main method in such a way that it must return a value to the caller, then it must return an integer type value and therefore you must specify return type to be int.

3.3 Function Types

We have two types of the functions one is the library function in c++ and user defined functions. Now, we are first going to discuss what are library functions, library functions are functions that are automatically included in the declared and used in the program, but that are not created by the users it is already present inside of the header file, which we always include at the beginning of the program.



Library Functions

- It is already present inside the header file which we always include at the beginning of a program.
- You just have to type the name of a function and use it along with the proper syntax.

User-defined Function

User-defined function is a type of function in which we have to write a body of a function and call the function whenever it's required.

3.4 Function Activities

Function activities are also divided into the function declaration, function definition, and function call.

1. Function declaration
2. Function definition
3. Function call

1. **Function Declaration:** Function declaration also called a function prototype. We just specify the name of a function that we are going to use in our program like a variable declaration.

Syntax:-

```
return_data_type function_name (data_type arguments);
```

2. **Function Definition:** Function definition means just writing the body of a function. What we are just going to write the body of the function, what kind of work we can do with the function, we are just writing the logic. The body of the function consists of the statements, which are going to be performed a specific task like a sub routine or a sub program, it is also known as the procedure. We are writing some of the lines there, we are writing some of the code there that is used to perform some of this specific task.

Function definition means just writing the body of a function. A body of a function consists of statements which are going to perform a specific task. Let's see definition of function is how implemented using code.

Syntax:-

```
int sum(int a,int b)
{
    int c;
    c=a+b;
    return c;
}
```

3. **Function Call:** While creating a C++ function, you give a definition of what the function has to do. To use a function, you will have to call or invoke that function.

Function call has two methods.

1. Call by value
2. Call by reference

1. **Call by value:** This method of passing arguments to a function copies the actual value of an argument into the formal parameter of the function. By default, C++ uses call by value to pass arguments.
2. **Call by reference:** This method copies the reference of an argument into the formal parameter. Inside the function, the reference is used to access the actual argument used in the call. This means that changes made to the parameter affect the argument.



Example

```
#include<iostream>
using namespace std;
int sum(int x,int y)
{
    return x+y;
}
int main(){
    int a,b;
    cout<<"Enter two numbers";
    cin>>a>>b;
    cout<<"Sum of entered number using call by value is = "<<sum(a,b);
    return 0;
}
```

Output

```
Enter two numbers 35
25
Sum of entered number using call by value is = 60
Process returned 0 (0x0)    execution time : 2.413 s
Press any key to continue.
```

3.5 Inline function

Now we are talking about inline functions c++ inline functions and inline function is a function that is explained in line when it is invoked with us saving the time we are going to save the functions or time to the execution then there are multiple ways we are calling the different functions we know that we are using the multiple functions in one single code. Inline function is used to execute the function but it not makes again the space inside of the memory. This copies the function to the location of the function call in the compile time when we are going to compile our program, it is going to copy the location of the function at the compile time and may take the program execution faster if it is going to save the function location into the compile time.

An inline function is a function that is expanded in line when it is invoked thus saving time. This copies the function to the location of the function call in compile-time and may make the program execution faster. Inline function is request to compiler not a command.

Advantages of Inline Function

1. It also saves the overhead of push/pop variables on the stack when function is called.

Unit 03: Functions and Compile Time Polymorphism

2. Inline function may be useful (if it is small) for embedded systems because inline can yield less code than the function call preamble and return.

Disadvantages of Inline Function

1. Use too many inline functions then the size of the binary executable file will be large, because of the duplication of same code.
2. Too much inlining can also reduce your instruction cache hit rate.
3. Inline functions may not be useful for many embedded systems. Because in embedded systems code size is more important than speed.

**Example**

```
#include <iostream>
using namespace std;
inline int display_number(int n){
cout<<"Number is "<< n<<endl;
}
int main() {
display_number(50);
display_number(150);
display_number(200);
return 0;
}
```

Output

```
Number is 50
Number is 150
Number is 200

Process returned 0 (0x0)   execution time : 0.014 s
Press any key to continue.
```

C++ Objects as Function Arguments

We can pass objects to a function in a similar manner as passing regular arguments.

**Example**

```
#include <iostream>
using namespace std;
class demo {
public:
    int n=100;
```

```

char ch='A';

void disp(demo d){
    cout<<d.n<<endl;
    cout<<d.ch<<endl;
}

};

int main() {
    cout<<"Passing object to function"<<endl;
    demo obj;
    obj.disp(obj);
    return 0;
}

```

Output

```

Passing object to function
100
A
Process returned 0 (0x0)   execution time : 0.289 s
Press any key to continue.

```

3.6 C++ Friend Functions

A friend function of a class is defined outside that the scope. And it can be accessible all the private and protected members of the class. And we're talking about the friend function in the friend function, when we declared a friend function in class, it can be accessed all of the members of the class which is private or protected and we are using the friend keyword inside the body of the class to declare the friend function.

Syntax

```

class className {
    ... ..
    friend returnType functionName(arguments);
    ... ..
}

```

3.7 Characteristics of friend function

1. Friend function is not in the scope of the class in which it has been declared as friend.
2. It allows to generate more efficient code.
3. It cannot be called using the object of the class as it is not in the scope of the class.
4. It can be called similar to a normal function without the help of any object.



Example

```
#include <iostream>
using namespace std;
class rectangle{
int a;
public:
    friend void disp(rectangle r);
    void get_length(int l);
};
void rectangle::get_length(int l){
a=l;
}
void disp( rectangle r){
cout<<"Entered length of rectangle is"<<r.a;
}
main(){
rectangle r;
r.get_length(10);
disp(r);
}
```

Output

```
Entered length of rectangle is10
Process returned 0 (0x0)   execution time : 0.006 s
Press any key to continue.
```

3.8 C++ Static Data Members & Functions

Inside a class definition, the keyword static declares members that are not bound to class instances. A static member is shared by all objects of the class. A static member is shared by all of the object of the class. Static member is the one of the member which can be shared with all of the objects in my class. Let us see the syntax of the static member function inside of the class.

Syntax

```
Class class_name{
private:
static data_member;
public:
static return_type function_name()
{
```

```
//body
    }
};
```



Example

```
#include <iostream>
using namespace std;
class Demo
{
    private:
        static int a;
    public:
        static void fun()
        {
            cout << "Value of a: " << a << endl;
        }
};
int Demo :: a =500;
int main()
{
    Demo obj;

    obj.fun();

    return 0;
}
```

Output

```
Value of a: 500

Process returned 0 (0x0)   execution time : 0.142 s
Press any key to continue.
```

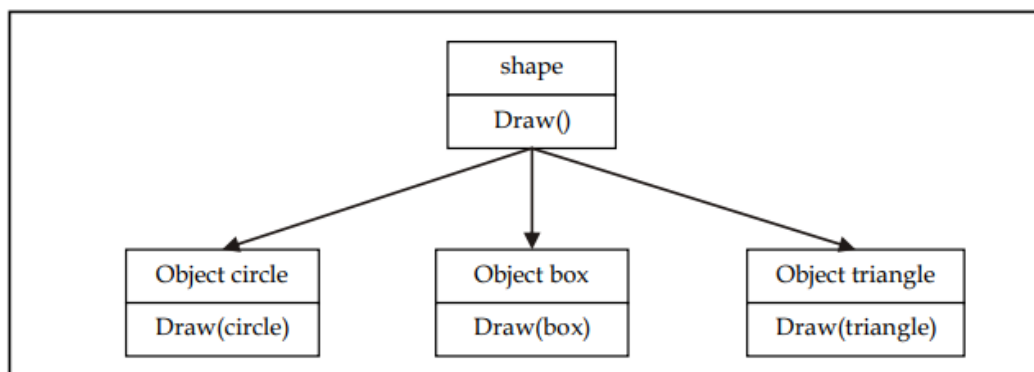
3.9 Polymorphism in C++

Polymorphism is an important OOP concept. Polymorphism means the ability to take more than one form. For example, an operation may exhibit different behavior in different instances. The behavior depends upon the types of data used in the operation. For example, consider the operation of addition. For two numbers, the operation will generate a sum. If the operands are strings, then the operation will produce a third string by concatenation. The diagram given below, illustrates that a single function name can be used to handle different number and types of

Unit 03: Functions and Compile Time Polymorphism

arguments. This is something similar to a particular word having several different meanings depending on the context.

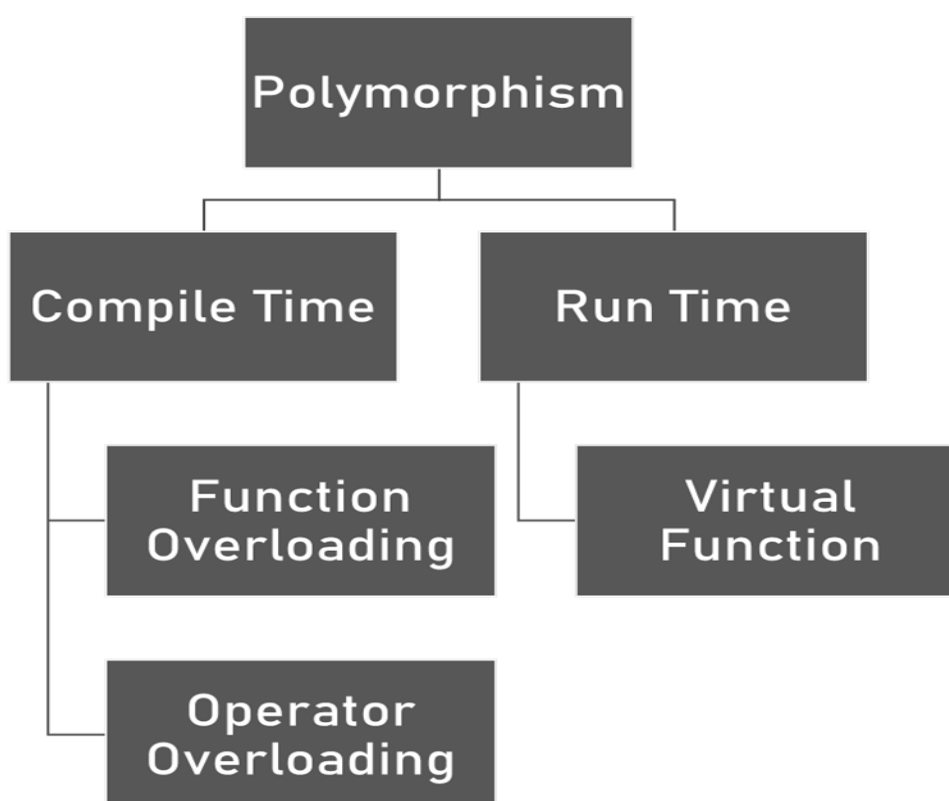
Polymorphism plays an important role in allowing objects having different internal structures to share the same external interface. This means that a general class of operations may be accessed in the same manner even though specific actions associated with each operation may differ. Polymorphism is extensively used in implementing inheritance as shown below.



Polymorphism can be implemented using operator and function overloading, where the same operator and function works differently on different arguments producing different results. These polymorphisms are brought into effect at compile time itself, hence is known as early binding, static binding, static linking or compile time polymorphism.

3.10 Types of C++ Polymorphism

1. Compile time Polymorphism
2. Runtime Polymorphism



3.11 Function Overloading

A function may take zero or more arguments when called. The number and type of arguments that a function may take is defined in the function itself. If a function call fails to comply by the number and type of argument(s), the compiler reports the same as error. Suppose we write a function named sum to add two numerical values given as arguments. One can write the function as:

```
int sum(int a, int b)
{
    return (a + b);
}
```

Now suppose we want the function to take float type argument then the function definition must be changed as:

```
float sumfloat(float a, float b)
{
    return (a + b);
}
```

As a matter of fact the function sum may take so many names as shown below.

```
int sumint(int a, int b)
{
    return (a + b);
}

short sumshort(short a, short b)
{
    return (a + b);
}

long sumlong(long a, long b)
{
    return (a + b);
}

float sumdouble(double a, double b)
{
    return (a + b);
}
```

This can be very tiring and extremely difficult to remember all the names. Function overloading is a mechanism that allows a single function name to be used for different functions. The compiler does the rest of the job. It matches the argument numbers and types to determine which functions is being called. Thus we may rewrite the above listed functions using function overloading as:

```
int sum(int a, int b)
{
    return (a + b);
}

float sum(float a, float b)
{
    return (a + b);
}
```

```

return (a + b);
}
short sum(short a, short b)
{
return (a + b);
}
long sum(long a, long b)
{
return (a + b);
}
float sum(double a, double b)
{
return (a + b);
}

```

Overloaded functions have the same name but different number and type of arguments. They can differ either by number of arguments or type of arguments or both. However, two overloaded function cannot differ only by the return type.



Example

```

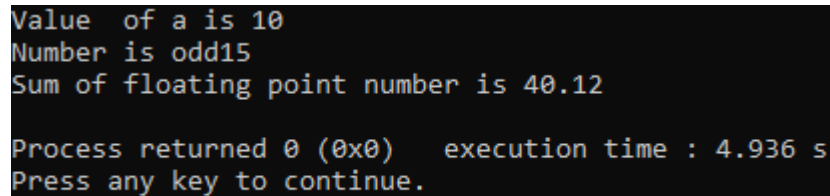
#include<iostream>
using namespace std;
class sample{
    public:
int chk_num(){
int a=10;
cout<<"Value of a is "<< a<<endl;
}
int chk_num(int a){
if(a%2==0)
    cout<<"Number is even" << a <<endl;
else
    cout<<"Number is odd" << a <<endl;
}
float chk_num(float x, float y)
{
cout<<"Sum of floating point number is "<< x+y<<endl;
}

};
main(){

```

```
sample obj;  
obj.chk_num();  
obj.chk_num(15);  
obj.chk_num(15.12,25);  
  
}
```

Output



```
Value of a is 10  
Number is odd15  
Sum of floating point number is 40.12  
  
Process returned 0 (0x0)   execution time : 4.936 s  
Press any key to continue.
```

Summary

An application written in C++ may have a number of classes. One of these classes must contain one (and only one) method called main method. Although a private main method is permissible in C++ it is seldom used. For all practical purposes the main method should be declared as public method. A function may take zero or more arguments when called. The number and type of arguments that a function may take is defined in the function itself. If a function call fails to comply by the number and type of argument(s), the compiler reports the same as error. When any program is compiled the output of compilation is a set of machine language instructions, which is in executable program. When a program is run, this complied copy of program is put into memory. C++ functions can have arguments having default values. The default values are given in the function prototype declaration. Prototype of a function is the function without its body. The C++ compiler needs to about a function before you call it, and you can let the compiler know about a function is two ways - by defining it before you call it or by specifying the function prototypes before you call it.

Keywords

Function: The best way to develop and maintain a large program is to divide it into several smaller program modules of which are more manageable than the original program. Modules are written in C++ as classes and functions. A function is invoked by a function call. The function call mentions the function by name and provides information that the called function needs to perform its task.

Function Declaration: Statement that declares a function's name, return type, number and type of its arguments.

Function Overloading: In C++, it is possible to define several function with the same name, performing different actions. The functions must only differ in their argument lists. Otherwise, function overloading is the process of using the same name for two or more functions.

Function Prototype: A function prototype declares the return-type of the function that declares the number, the types and order of the parameters, the function expects to receive. The function prototype enables the compiler to verify that functions are called correctly.

Inline Function: A function definition such that each call to the function is, in effect, replaced by the statements that define the function.

Self Assessment

1. Which of the following function / types of function cannot have default parameters?
 - A. Member function of class
 - B. Main()
 - C. Member function of structure
 - D. None of above

Unit 03: Functions and Compile Time Polymorphism

2. Function call type is
 - A. Call by value
 - B. Call by reference
 - C. All of Above
 - D. None of Above

3. Which of the following is the default return value of functions in C++?
 - A. int
 - B. char
 - C. float
 - D. void

4. Choose the correct statement for call by value
 - A. Copy of variable is passed.
 - B. Original value not modified.
 - C. Actual arguments remain safe as they cannot be modified accidentally.
 - D. All of Above

5. What is an inline function?
 - A. A function that is expanded at each call during execution
 - B. A function that is called during compile time
 - C. A function that is not checked for syntax errors
 - D. A function that is not checked for semantic analysis

6. An inline function is expanded during _____
 - A. compile-time
 - B. run-time
 - C. never expanded
 - D. end of the program

7. What are the two advantage of function objects than the function call?
 - A. It contains a state
 - B. It is a type
 - C. It contains a state & It is a type
 - D. It contains a prototype

8. Pick out the correct statement.
 - A. A friend function may be a member of another class
 - B. A friend function may not be a member of another class
 - C. A friend function may or may not be a member of another class
 - D. None of the mentioned

9. Where does keyword 'friend' should be placed?
 - A. function declaration
 - B. function definition
 - C. main function
 - D. block function

10. Which keyword is used to declare the friend function?
 - A. firend
 - B. friend
 - C. classfriend
 - D. myfriend

11. Which keyword should be used to declare static variables?
 - A. static
 - B. stat

- C. common
- D. const

12. Which is the correct syntax for declaring static data member?

- A. static mName dataType;
- B. dataType static mName;
- C. mName static dataType;
- D. static dataType mName;

13. Overloaded functions in C++ oops are

- A. Functions preceding with virtual keywords.
- B. Functions inherited from base class to derived class.
- C. Two or more functions having same name but different number of parameters or type.
- D. None of above

14. Function overloading is _____ in C++.

- A. Class
- B. Object
- C. Compile Time Polymorphism
- D. None of above

15. What is the output of this program?

```
#include <iostream>

using namespace std;

int Add(int X, int Y, int Z)
{
    return X + Y;
}

double Add(double X, double Y, double Z)
{
    return X + Y;
}

int main()
{
    cout << Add(5, 6);
    cout << Add(5.5, 6.6);
    return 0;
}
```

- A. 11 12.1
- B. 12.1 11
- C. 11 12
- D. Compile time error

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. B | 2. C | 3. A | 4. D | 5. A |
| 6. A | 7. C | 8. C | 9. A | 10. B |
| 11. A | 12. D | 13. C | 14. C | 15. D |

Review Questions

1. What is function prototyping? Why is it necessary? When is it not necessary?
2. What is purpose of inline function?
3. Differentiate between the following:
 - (a) void main()
 - (b) int main()
 - (c) int main(int argn, char argv[])
4. In which cases will declare a member function to be friend?
5. Write a program that uses overloaded member functions for converting temperature from Celsius to Kelvin scale.
6. To calculate the area of circle, rectangle and triangle using function overloading.
7. Do inline functions improve performance?
8. How can inline functions help with the tradeoff of safety vs. speed?
9. Write a program that demonstrate working of function overloading.
10. Write a program that demonstrates working of inline functions.

**Further Readings**

- E Balagurusamy; Object Oriented Programming with C++; Tata Mc Graw-Hill.
- Herbert Schildt; The Complete Reference C++; Tata Mc Graw Hill. Robert Lafore;
- Object-oriented Programming in Turbo C++; Galgotia.

**Web Links**

- https://en.wikipedia.org/wiki/Object-oriented_programming
- www.web-source.net
- www.webopedia.com